

SUMMARY

본 문서는 기술 평가 보고서가 아니라 Agent 과제 제출 전 구조·요구사항 점검 보고서다. 실제 기술 평가 결과 보고서는 기존 보고서 형식을 유지해 별도 PDF로 제출해야 한다. 본 점검에서는 기존 팀 구현의 기술 선정, 논문 RAG, 외부 검색, 네 개 평가 관점을 유지하면서 Orchestrator-Workers 패턴을 적용했는지 확인한다. 실행 시점에 Orchestrator가 input_request의 관점과 재시도 대상을 읽어 2·3·4개 subtask 계획을 만들고 Send로 동적 fan-out한다. Worker 결과는 task_id 기준 reducer로 합쳐져 synthesizer로 전달된다. Worker 실패는 continue_with_limitations 정책으로 기록하고, 합성은 계속하되 누락을 State와 보고서 한계에 남긴다. 보고서 생성 이후 quality evaluator는 Groundedness, 필수 목차, 중립성, 편향 통제, 관점 커버리지를 검사한다. 품질 미달이면 조사 문제는 해당 Worker를, 문서 문제는 report generation을 재실행하며 반복 상한 초과 시 warning 종료한다. 실제 논문 다운로드·Ollama 임베딩·OpenAI 호출과 LangSmith 캡처는 인증·로컬 환경에 의존하므로 이 문서의 구조 검증과 구분한다.

1. 분석 배경

KV cache는 긴 문맥과 동시 요청이 늘어날수록 LLM 서빙 메모리 요구량과 연결된다. 팀의 기존 RAG 보고서는 이 병목을 모델 구조에서 저장량을 줄이는 MLA와 GPU 외부의 계층형 메모리를 활용하는 ITME라는 서로 다른 계층의 접근으로 나누어 평가했다. 두 기술은 동일한 조건에서 직접 벤치마크한 대상이 아니므로, 공개 논문과 외부 자료의 실험 조건을 함께 기록하고 수치를 단순 우열 비교하지 않는 원칙을 유지한다.

이번 변경의 목표는 기술 결론을 새로 만드는 것이 아니라, 기존 조사 Agent가 과제의 동적 협업 요구를 만족하도록 조정 계층을 보완하는 것이다. 따라서 기술별 원리·성능·한계·TRL, 시장성, 이해관계자, 도메인 적용성이라는 기존 역할 분리는 유지하고 계획·fan-out·fallback·품질 Loop를 Graph와 State에 명시했다.

2. 기술 선정

소프트웨어 대상은 DeepSeek-V2의 Multi-head Latent Attention(MLA)이다. MLA는 KV 표현을 latent representation으로 압축하는 모델 구조 기반 접근으로 설명되며, DeepSeek-V2 원문은 비교 조건에서 KV cache 절감량과 생성 처리량 결과를 보고한다. 하드웨어 대상은 ITME(Inference Tiered Memory Expansion with Disaggregated CXL-Hybrid Memories)다. ITME는 KV cache 표현을 직접 줄이기보다 CXL-Hybrid memory를 이용해 메모리 계층을 확장하는 시스템 접근으로 정의된다.

두 대상은 서로 다른 계층을 개입하므로 같은 숫자 하나로 승패를 판정하지 않는다. 기술 성숙도에서는 논문 실험·프로토타입·공개 배포·운영 근거를 분리하고, 시장성과 이해관계자 관점에서는 직접 채택 근거와 주변 생태계 근거를 구분한다. 공개 자료가 부족한 항목은 추정으로 채우지 않고 공개 정보 부족과 후속 검증 질문으로 남긴다.

3. Orchestrator-Workers 적용

orchestrator_plan_node가 input_request에 포함된 관점 키워드와 재시도 대상에 따라 plan을 만든다. 기술 성숙도·도메인 질문은 2개, 시장성까지 포함하면 3개, 네 관점을 모두 요구하면 4개 계획이 생성된다. 각 항목은 task_id, agent, objective, status, attempt를 가진 구조화된 객체다. fan_out_or_warn는 계획의 실제 길이만큼 Send("worker", ...)를 생성하므로 실행 요청에 따라 Worker 수가 달라진다. Worker는 다른 Worker를 호출하지 않고 각자 결과를 반환한다.

Fan-in 이후 synthesis가 관점 결과를 종합한다. Worker 예외나 빈 분석 결과는 실패 결과와 오류로 기록하며 continue_with_limitations 정책을 적용한다. 이 정책은 부분 결과를 사용하되 누락을 숨기지 않는 fallback이다. max_steps와 MAX_RETRIES는 계획 재실행 횟수를 제한하여 무한 Loop를 막는다.

4. State Schema와 관측성

제어 정보와 페이로드를 구분했다. plan, current_task, task_results, decision_log, trace_id, status, step_count, retry_targets는 조정·복구를 위한 제어 메타데이터다. selected_technologies, 각 관점 분석, synthesis, references, report는 작업 결과와 근거 페이로드다. merge_task_results는 동적 fan-out에서 같은 결과 필드가 유실되지 않도록 task_id 기준으로 병합한다. 기존 merge_references는 논문 청크와 웹 URL 중복을 제거한다.

각 계획·fallback·품질 판정은 decision_log에 시간, 노드, 이벤트 유형, 메시지, 사유를 기록한다. trace_id는 State JSON과 실행 trace를 연결하는 키다. 최종 보고서 본문에는 가독성을 위해 내부 Evidence ID를 숨기되, Agent 결과와 REFERENCE에는 연결을 보존한다.

5. 품질 평가와 재작업

보고서 생성 뒤 quality_evaluator를 실행한다. 평가 항목은 Groundedness, 필수 목차(SUMMARY·REFERENCE), 중립성, 편향 통제, 네 관점 커버리지, Reference 연결성이다. 편향 통제는 관점별 서로 다른 출처가 2개 이상인지와 최대 단일 출처 비중이 50% 이하인지 확인한다. 현재 구현은 과제의 1인인 결정론적 rubric을 선택했으며, 인용 존재성·출처 다양성까지 코드로 검사해 단순 금지어 검사보다 강한 기준을 사용한다. LLM Judge의 확률적 판정을 라우팅 함수 안에 직접 넣지 않고, 구조화된 평가 결과를 결정론적 conditional edge로 연결한다.

평가가 실패하면 retry_targets에 해당 관점 Worker를 기록하고 Orchestrator가 그 대상만 다시 계획한다. 보고서가 통과하면 종료하고, 재시도

상한에 도달하면 pass_with_limitations와 warning 상태로 종료한다. 따라서 정상 종료, 제한사항 종료, 반복 상한 종료가 모두 명시돼 있다.

6. 검증 결과

API 키와 Ollama 실행 상태에 의존하지 않는 구조 검증에서 Graph는 요청 관점에 따라 2·3·4개 subtask를 계획하고 Worker fan-out/fan-in을 수행했다. 첫 품질 평가를 실패시키는 mock 시나리오에서는 technical_research와 domain_evaluation만 재계획한 뒤 두 번째 평가에서 통과했다. 추가로 편향 통제, 빈 근거·추천 표현 검출, report-only retry, synthesis 재시도 경로, 런타임 요청 입력을 테스트했다. 현재 tests/test_smoke.py에는 총 17개 테스트가 있다.

이 검증은 실제 논문 다운로드, Chroma 색인, Ollama 임베딩, Tavily/DDGS 검색, OpenAI 보고서 생성을 대신하지 않는다. 제출 전에는 기존 팀 보고서와 동일한 목차의 새 Agent 실행 보고서를 FAST_MODE=false로 생성하고, 한글 글꼴이 포함된 PDF·State JSON·LangSmith 화면 trace를 함께 확인해야 한다. 현재 docs/local_trace.png는 LangSmith 캡처가 아니므로 docs/tracing-1.png, docs/tracing-2.png를 별도로 추가해야 한다.

REFERENCE

- 팀 제공 RAG 설계서: RAG-Design_판교캠퍼스-7반_....pdf
- 팀 제공 이전 RAG 보고서: RAG-Output_판교캠퍼스-7반_....pdf
- DeepSeek-V2: <https://arxiv.org/abs/2405.04434>
- ITME: <https://arxiv.org/abs/2606.12556>
- LangGraph Send API: <https://docs.langchain.com/oss/python/langgraph/graph-api>